

Titulación: Grado en Ingeniería Informática, Ingeniería en Sistemas de Información e InfoADE
Curso: 2024-2025. Convocatoria Ordinaria de Junio
Asignatura: Bases de Datos Avanzadas – Laboratorio

Practica 4: Replicación e Implementación de una Base de Datos Distribuida.

ALUMNO 1:

Nombre y Apellidos: Gonzalo López Menor

DNI: 03210671D

ALUMNO 2:

Nombre y Apellidos: Juan Cartagena Castro

DNI: 09101498F

Fecha: 4/05/2025

Profesor Responsable: Maria Teresa Villalba

Mediante la entrega de este fichero los alumnos aseguran que cumplen con la normativa de autoría de trabajos de la Universidad de Alcalá, y declaran éste como un trabajo original y propio.

En caso de ser detectada copia, se puntuará **TODA** la asignatura como Suspense – Cero.

Plazos

Tarea en laboratorio: Semana 21 de abril, Semana 28 de abril, Semana 5 de mayo y semana 12 de mayo.

Entrega de práctica: **Día 16 de mayo.** Aula Virtual

Documento a entregar: Este mismo fichero con la implementación de la replicación y la base de datos distribuida, las pruebas realizadas de su funcionamiento; y los ficheros de configuración del maestro y del esclavo utilizados en replicación; y de la configuración de los servidores de la base de datos distribuida (obligatorio). También los ficheros de log generados en la realización de la práctica de los postgres involucrados (obligatorio). Fichero para subir **DNI'sdelosAlumnos_PECL4.zip**

Los profesores del laboratorio podrán solicitar una presentación/defensa de la práctica para ver que todo funciona correctamente.

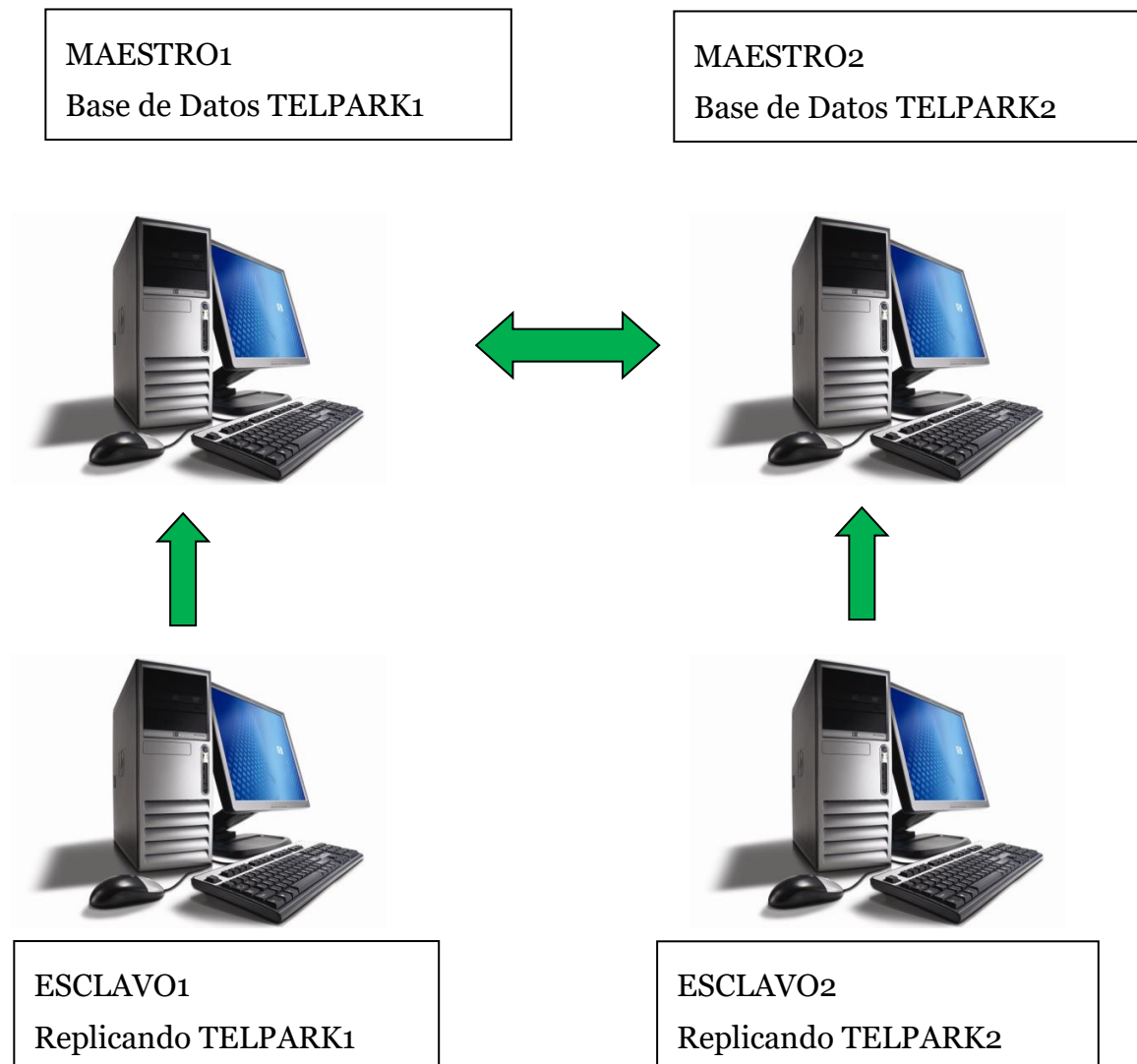
AMBOS ALUMNOS DEBEN ENTREGAR EL FICHERO EN LA PLATAFORMA.

Introducción

El contenido de esta práctica versa sobre la Replicación de Bases de Datos con PostgreSQL e introducción a las bases de datos distribuidas. Concretamente se va a utilizar los servicios de replicación de bases de datos que tiene PostgreSQL. Para ello se utilizará PostgreSQL 17.x con soporte para replicación. **Se prohíbe el uso de cualquier otro programa externo a PostgreSQL para realizar la replicación, como puede ser Slony.**

También se va a diseñar e implementar una pequeña base de datos distribuida. Una base de datos distribuida es una base de datos lógica compuesta por varios nodos (equipos) situados en un lugar determinado, cuyos datos almacenados son diferentes; pero que todos ellos forman una base de datos lógica. Generalmente, los datos se reparten entre los nodos dependiendo de donde se utilizan más frecuentemente.

El escenario que se pretende realizar se muestra en el siguiente esquema:



Se van a necesitar 4 postgres: 2 maestros y 2 esclavos. Cada maestro puede ser un ordenador de cada miembro del grupo con una base de datos de TELPARK en concreto (TELPARK1 y TELPARK2). Dentro de cada maestro se puede instalar una máquina virtual, que se corresponderá con el esclavo que se encarga de replicar la base de datos que tiene cada maestro, es decir, hace una copia o backup continuo de la base de datos TELPARK1 o de la base de datos TELPARK2. También es posible no usar otra máquina virtual para el esclavo y levantar otra estancia de postgres para guardar la réplica en un clúster diferente. Hay más de una solución.

Se debe de entregar el cuestionario que posea los siguientes puntos para poder ser reproducido el trabajo por cualquier persona que pueda leerlo.

Cuestión 0. Configurar el fichero de Error Reporting and Logging de PostgreSQL para que aparezcan recogidas las sentencias SQL DDL (Lenguaje de Definición de Datos) + DML (Lenguaje de Manipulación de Datos) generadas en dicho fichero. No se pide activar todas las sentencias. No activar la duración de la consulta. También se debe de configurar el log para que en el comienzo de la línea de registro de la información del log ("line prefix") aparezcan los DNI's de los miembros del equipo, el nombre del host con su puerto y la **fecha y hora** de la operación.

Nos vamos a la ruta correspondiente donde esté el archivo postgres.conf

En este caso: C:\Program Files\PostgreSQL\16\data\log

Y modificamos el parámetro log_statement estableciéndolo en ' mod '

Después hay que modificar el prefijo de la línea de registro line_prefix incluyendo el dni, nombre host, el puerto y la fecha y hora (%t). Esto se hace mirando lo que nos pone el archivo .conf en cuanto a instrucciones.

La duración de consulta la desactivamos con log_duration = off

Acceso a Bases de Datos Remotas usando el método del Apéndice F36. Postgres_fdw

Cuestión 1. Configuración de cada uno de los nodos maestros de la base de datos de TELPARK1 y TELPARK2 por medio del **módulo postgres_fdw** que se puedan recibir y realizar consultas sobre las tablas de la base de datos que no tienen implementadas localmente.

```
CREATE SCHEMA IF NOT EXISTS remoto;
```

Vamos a optar por no usar maquina virtual y crear dos estancias.

Lo primero que hacemos es crear las dos instancias con initdb.exe -D "C:\pg_data\TELPARK1" -U postgres y lo mismo con TELPARK2 (o creamos dos carpetas y empleamos pg_basebackup).

Tras esto, configuramos los postgresql.conf y los pg_hba.conf.

En postgresql.conf cambiamos los puertos por 5433 y 5434 correspondientes y tambien habilitamos listen_addresses en local. Con el pg_hba.conf, nos fijamos que abajo del todo tiene el apartado # Allow replication connections from localhost, by a user with the # replication privilege, no lo tocamos.

Según el manual, para iniciar un servidor se puede hacer con:

postgres -D /usr/local/pgsql/data (adaptado a nuestra ruta) asi que iniciamos los dos.

O tambien podemos iniciarlo de la siguiente manera: pg_ctl -D C:/TELPARK1

En TELPARK2 creamos el rol fdw CREATE ROLE fdw_user WITH LOGIN;

GRANT CONNECT ON DATABASE telpark2 TO fdw_user;

GRANT USAGE ON SCHEMA public TO fdw_user;

GRANT SELECT ON ALL TABLES IN SCHEMA public TO fdw_user;

ALTER DEFAULT PRIVILEGES IN SCHEMA public

GRANT SELECT ON TABLES TO fdw_user;

y creamos las tablas vacías originarias de telpark

En TELPARK1, creamos la extension fdw CREATE EXTENSION IF NOT EXISTS postgres_fdw;

y creamos el servidor CREATE SERVER telpark2_srv

FOREIGN DATA WRAPPER postgres_fdw

OPTIONS (

host '127.0.0.1',

port '5434',

dbname 'telpark2'

);

Por otro lado, creamos el user mapping CREATE USER MAPPING FOR postgres

SERVER telpark2_srv

OPTIONS (user 'fdw_user');

y el esquema remoto:

Para acabar importando las tablas en al base de datos TELPARK1 remotamente IMPORT FOREIGN SCHEMA public

LIMIT TO (clientes, vehiculos, plazas, reservas, pagos, incidencias)

FROM SERVER telpark2_srv

INTO remoto;

Ahora si queremos desde la base de datos TELPARK1 consultar un cliente, podemos hacer SELECT * FROM remoto.clientes;

Cuestión 2. Insertar datos diferentes en cada una de las bases de datos del MAESTRO1 y del MAESTRO2. Realizar una consulta sobre el MAESTRO1 que permita obtener el número total de reservas de los clientes de cada provincia mostrando provincia y número total de reservas. Se debe de usar toda la base de datos distribuida (MAESTRO1 + MAESTRO2) en una sola consulta SQL. Explicar cómo se resuelve la consulta y su plan de ejecución.

Consulta

```
EXPLAIN ANALYZE SELECT provincia, COUNT(*) AS total_reservas FROM (SELECT c.provincia
FROM clientes c JOIN reservas r ON r.clienteid_clientes = c.clienteid UNION ALL SELECT
c.provincia FROM remoto.clientes c JOIN remoto.reservas r ON r.clienteid_clientes = c.clienteid )
AS todas GROUP BY provincia;
```

Plan de Ejecución

```
QUERY PLAN
-----
HashAggregate (cost=1971.00..1973.00 rows=200 width=40) (actual time=0.635..0.637 rows=4
loops=1)
  Group Key: c.provincia
  Batches: 1 Memory Usage: 40kB
  -> Append (cost=21.02..1864.39 rows=21323 width=32) (actual time=0.039..0.631 rows=4 loops=1)
    -> Hash Join (cost=21.02..48.23 rows=1360 width=32) (actual time=0.036..0.038 rows=2
loops=1)
      Hash Cond: (r.clienteid_clientes = c.clienteid)
      -> Seq Scan on reservas r (cost=0.00..23.60 rows=1360 width=4) (actual time=0.013..0.014
rows=2 loops=1)
        -> Hash (cost=14.90..14.90 rows=490 width=36) (actual time=0.012..0.012 rows=2 loops=1)
          Buckets: 1024 Batches: 1 Memory Usage: 9kB
          -> Seq Scan on clientes c (cost=0.00..14.90 rows=490 width=36) (actual
```

Explicaciones y Comentarios

La consulta se descompone internamente en varias fases bien coordinadas. Primero, PostgreSQL realiza un escaneo secuencial (Seq Scan) sobre la tabla clientes del nodo local, extrayendo cada fila y leyendo la columna provincia. Para cada cliente detectado, se emplea un bucle anidado (Nested Loop) que activa un escaneo por índice (Index Scan) en la tabla reservas, filtrando únicamente las filas cuyo campo clienteid_clientes coincida con la clave primaria del cliente actual. De este subproceso local surge un conjunto inicial de provincias de las reservas registradas en el maestro.

Paralelamente, la parte del plan correspondiente al Foreign Data Wrapper traduce esa misma lógica de join en un bloque de SQL remoto que se envía al esclavo. Allí, el servidor secundario ejecuta de manera autónoma el join entre sus tablas clientes y reservas, proyecta únicamente la columna provincia y devuelve ese flujo de datos ya reducido al maestro. Este push down remoto minimiza el tráfico de red, porque únicamente circulan las provincias en lugar de todas las columnas o filas completas.

Una vez recibidas ambas corrientes —la local y la remota— el plan las concatena con un nodo Append, uniendo ambos orígenes sin alterar su contenido. A continuación, un Sort ordena el conjunto completo por provincia, preparando los datos para la agregación. Finalmente, un Finalize Aggregate agrupa las filas por provincia y aplica COUNT(*), computando el número total de reservas para cada territorio.

Replicación Física usando método capítulo 26 de replicación.

Cuestión 3. Configuración completa de los equipos para estar en modo de replicación. Configuración del nodo maestro y del nodo esclavo. Tipos de nodos maestros, diferencias en el modo de funcionamiento y tipo elegido. Tipos de nodos esclavos,

Preparamos el nodo maestro(TELPARK1) editando postgresql.conf con wal_level en replica, activando el envío de WAL.

En pg_hba.conf habilitamos la conexión de replica desde localhost.

y creamos el usuario de replicación. CREATE ROLE repuser WITH REPLICATION

ALTER ROLE repuser WITH LOGIN;

Para preparar el nodo esclavo, eliminamos todo lo que hubiera dentro de TELPARK2

y usamos pg_basebackup como repuser para copiar todo, creando un archivo de standby.signal.

pg_basebackup -h 127.0.0.1 -p 5433 -U repuser -D C:\pg_data\TELPARK2 -X stream -P -R

Una vez hecho esto, se habrá completado la replicación.

SELECT pid, state, sync_state

FROM pg_stat_replication;

y nos sale pid | state | sync_state

-----+-----+-----

8616 | streaming | async

(1 fila)

Con esto podemos ver que existen varios tipos de nodos maestros: Single primary, donde un unico nodo acepta escrituras y genera WAL. Primary sincrono, que bloquea el commit hasta que al menos un standby confirma haber recibido el WAL, y cascading primary, que el primary envía WAL a un primer nivel de standys que a su vez pueden enviar a otros.

Por otro lado estan los tipos de nodos esclavos/standby:

Warm Standby que solo aplica WAL y no puede atender consultas, el Hot standby que aplica WAL y permite realizar consultas de solo lectura y Cascading Standby que recibe Wal de otro standby en lugar de hacerlo directamente del primary. Los elegidos han sido Single primary asincrono por su sencillez y cumpliendo de la práctica sin añadir latencia y por otro lado el hot standby, para poder verificar la replica haciendo select's..

Para poder verificar, nos conectamos desde el nodo esclavo a telpark1 y ejecutamos SELECT * FROM clientes, viendo que efectivamente obtiene las filas que habiamos insertado en el primary

Insertamos desde el nodo maestro: telpark1=# INSERT INTO clientes VALUES (4,4,4,4,4);

INSERT o 1

telpark1=#

y consultamos desde el nodo esclavo: telpark1=# SELECT * FROM clientes;

clienteid | nombre | apellido | email | telefono | provincia

-----+-----+-----+-----+-----+-----

1 | Pedro | Martínez | pedro@example.com | 600100100 | Sevilla

2 | Laura | Sánchez | laura@example.com | 600100101 | Bilbao

4 | 4 | 4 | 4 | 4 | 4

(3 filas)

telpark1=#

Cuestión 4. Comentar los tipos de operaciones que se pueden realizar en cada tipo de nodo. Además, provocar situaciones de caída de los nodos y observar los mensajes obtenidos por cada nodo. Realizar las acciones correctoras necesarias para volver el sistema a un estado normal de operaciones. Comentar lo observado y realizado en esta cuestión.

Los tipos de operaciones permitidas que hay en el nodo maestro son DDL y DML, emplear backups y de monitoreo, mientras que las operaciones que hay permitidas dentro del nodo esclavo son de lectura, no permite escrituras de ningún tipo (telpark1=# CREATE TABLE mi_tabla_aleatoria (id SERIAL PRIMARY KEY, nombre VARCHAR(50), cantidad INTEGER, fecha_creacion DATE, valor DECIMAL(10, 2));

ERROR: no se puede ejecutar CREATE TABLE en una transacción de sólo lectura
telpark1=#)

Vamos a simular caída de los nodos y a observar los mensajes que han sido obtenidos por cada nodo. Para ello empleamos pg_ctl: pg_ctl stop -D C:\pg_data\TELPARK1 Simulando que el nodo primary se cae.

Y en el log_standby.log vemos estos errores:

DNI:03210671D DN2:09101498F 2025-05-04 18:14:35.071 CEST [23184] 2025-05-04 18:14:35

CESTFATAL: no se pudo conectar al servidor primario: falló la conexión al servidor en «127.0.0.1», puerto 5433: Connection refused (0x0000274D/10061)

¿Está el servidor en ejecución en ese host y aceptando conexiones TCP/IP?

DNI:03210671D DN2:09101498F 2025-05-04 18:14:35.071 CEST [24412] 2025-05-04 18:14:35

CESTLOG: waiting for WAL to become available at 0/3000738

DNI:03210671D DN2:09101498F 2025-05-04 18:14:40.087 CEST [25048] 2025-05-04 1

8:14:40 CESTFATAL: no se pudo conectar al servidor primario: falló la conexión al servidor en «127.0.0.1», puerto 5433: Connection refused (0x0000274D/10061)

¿Está el servidor en ejecución en ese host y aceptando conexiones TCP/IP?

DNI:03210671D DN2:09101498F 2025-05-04 18:14:40.087 CEST [24412] 2025-05-04 18:14:40

CESTLOG: waiting for WAL to become available at 0/3000738

Y además vemos que el nodo esclavo sigue en stand by: SELECT pg_is_in_recovery(); con salida true
Como acción correctora necesaria para volver el sistema a un estado normal de operaciones, promovemos el standby a primary: SELECT pg_promote(wait_seconds => 60); por lo que ahora se aceptan escrituras. Tras esto, reinstanciaríamos el antiguo primary como standby y se cambiarían los nodos. Eliminamos el directorio de datos de TELPARK1 (antiguo nodo maestro) y ejecutamos pg_basebackup desde usuario repuser como hicimos anteriormente con TELPARK2.

Arrancamos TELPARK1 como standby: pg_ctl start -D C:\pg_data\TELPARK1 -o "-p 5433" -l C:\pg_data\TELPARK1\log_standby.log

La segunda simulación que hacemos es que se cae el nodo esclavo (como ahora están cambiados, el correspondiente a TELPARK1).

Paramos el standby (TELPARK1) pg_ctl stop -D C:\pg_data\TELPARK1

Ejecutamos en TELPARK2 (nodo maestro) SELECT * FROM pg_stat_replication;

y vemos que walsender no existe, ha desaparecido.

La acción correctora sería reiniciar el standby, por lo que walreceiver se reconectará automáticamente y aplicará los WAL pendientes.

Replicación lógica usando el procedimiento del capítulo 29

Cuestión 5. Configuración completa de los equipos para estar en modo de replicación lógica. Configuración de ambos tipos de nodos, funcionamiento de ambos tipos de nodos, etc.

La replicación lógica usa un modelo de publish y suscripción con uno o mas suscriptores suscribiéndose a una o mas 'publicaciones' en un nodo publisher. Una vez que se ha hecho esto, los suscriptores extraen datos de las publicaciones a las que se suscriben y luego pueden volver a publicar datos para permitir la replicación en cascada u otras configuraciones.

Para ello, cambiamos el parámetro del publisher(que será TELPARK1) de wal_level que lo ponemos a logical. hot_standby en on del subscriber. Además creamos la publicación para las tablas clientes y reservas: CREATE PUBLICATION telpark_pub FOR TABLE clientes, reservas;

Creamos tablas vacías que tengan la misma estructura.

Y creamos la suscripción que apunte al Publisher:

```
CREATE SUBSCRIPTION telpark_sub
```

```
CONNECTION 'host=127.0.0.1 port=5433 user=repuser dbname=telpark1'
```

```
PUBLICATION telpark_pub;
```

Esto toma una snapshot inicial de clientes y reservas y aplica en tiempo real todos los DML que se hagan en TELPARK1

Insertamos en el publisher: INSERT INTO clientes VALUES

```
(15,'publisher','publisher','publisher',600000000,'Madrid');
```

Y vemos en subscriber.

```
telpark1=# SELECT * FROM clientes;
```

```
clienteid | nombre | apellido | email | telefono | provincia
```

```
-----+-----+-----+-----+-----+-----
```

```
1 | Pedro | Martínez | pedro@example.com | 600100100 | Sevilla
```

```
2 | Laura | Sánchez | laura@example.com | 600100101 | Bilbao
```

```
4 | 4 | 4 | 4 | 4 | 4
```

```
15 | publisher | publisher | publisher | 600000000 | Madrid
```

```
500 | X | Y | x@y.com | 600123123 | Toledo
```

(5 filas)

```
telpark1=#
```

Con esto, cualquier inserción que se haga en el publisher saldrá en el subscriber inmediatamente.

Como conclusión, el Publisher registra en WAL todos los cambios de las tablas de la publicación (wal_level = logical).

El Subscriber inicia un worker que conecta, toma el snapshot inicial y luego aplica cada DML que llega.

Cuestión 6. Comentar los tipos de operaciones que se pueden realizar en cada tipo de nodo. Además, provocar situaciones de caída de los nodos y observar los mensajes obtenidos por cada nodo. Realizar las acciones correctoras necesarias para volver el sistema a un estado normal de operaciones. Comentar lo observado y realizado en esta cuestión.

Las operaciones permitidas en cada tipo de nodo son las siguientes:

En el nodo Publisher se permiten todo tipo de creación de tablas, consultas, monitoreo, gestión de slots, creación de publicaciones... etc. Mientras que en el nodo Suscriber, solo es posible hacer SELECT sobre las tablas replicadas, DML en las tablas no publicadas, y create subscription.

Ahora vamos a simular que se cae el publisher parando el servidor.

Observamos mensajes como estos: ¿Está el servidor en ejecución en ese host y aceptando conexiones TCP/IP?

DNI:03210671D DN2:09101498F 2025-05-04 19:23:17.358 CEST [25796] 2025-05-04 19:23:17

CESTLOG: proceso ayudante «logical replication apply worker» (PID 6568) terminó con código de salida 1

DNI:03210671D DN2:09101498F 2025-05-04 19:23:20.331 CEST [26532] 2025-05-04 19:23:20

CESTLOG: el ayudante «apply» de replicación lógica para la suscripción «telpark_sub» ha iniciado

DNI:03210671D DN2:09101498F 2025-05-04 19:23:22.355 CEST [26532] 2025-05-04 19:23:22

CESTERROR: no se pudo conectar con el publicador: falló la conexión al servidor en «127.0.0.1», puerto 5433: Connection refused (0x0000274D/10061)

¿Está el servidor en ejecución en ese host y aceptando conexiones TCP/IP?

DNI:03210671D DN2:09101498F 2025-05-04 19:23:22.359 CEST [25796] 2025-05-04 19:23:22

CESTLOG: proceso ayudante «logical replication apply worker» (PID 26532) terminó con código de salida 1

DNI:03210671D DN2:09101498F 2025-05-04 19:23:25.339 CEST [7052] 2025-05-04 19:23:25

CESTLOG: el ayudante «apply» de replicación lógica para la suscripción «telpark_sub» ha iniciado

Las acciones correctoras necesarias serían simplemente iniciar de nuevo al publisher y verificar que la suscripción está activa con SELECT subscription_name, state

FROM pg_stat_subscription;

Si se cae el suscriber, lo volvemos a iniciar y comprobamos que el worker lógico aplica los cambios pendientes. SELECT *

FROM pg_stat_subscription

WHERE subscription_name = 'telpark_sub';

Si no es así, hacemos un reset de la suscripción con: ALTER SUBSCRIPTION telpark_sub REFRESH PUBLICATION;

Cuestión 7. Conclusiones.

En esta práctica hemos aprendido a integrar datos distribuidos con postgres_fdw, permitiendo consultar tablas de un nodo remoto sin duplicar información; a configurar replicación física en streaming estableciendo un primary y un hot standby mediante pg_basebackup y parámetros como wal_level, max_wal_senders y hot_standby, así como a gestionar la promoción y reincorporación de nodos tras fallos; a implementar replicación lógica con publicaciones y suscripciones, manejando slots, permisos SELECT y snapshots iniciales para replicar solo las tablas deseadas; y a simular caídas de ambos tipos de nodo, analizarlas en vistas como pg_stat_replication y pg_stat_subscription y restaurar el servicio con comandos como pg_promote, DROP/CREATE SUBSCRIPTION y ajustes en pg_hba.conf. Destacamos la importancia de otorgar privilegios adecuados, coordinar puertos y métodos de autenticación, y usar cada método de réplica según necesitemos alta disponibilidad o replicación selectiva, consolidando así una arquitectura de PostgreSQL robusta y escalable. Por último se puede ver el proceso que se realiza al caerse una base de datos tanto como si se cae el nodo maestro como el nodo esclavo en general.

Se deben mostrar evidencias de que ha funcionado el sistema, por eso se deben de entregar los logs y ficheros de configuración de cada equipo utilizados en la práctica.

Bibliografía

- Capítulo 18: Server Setup and Configuration.
- Capítulo 29: Server Configuration.
- Capítulo: 20.1. The pg_hba.conf File
- Capítulo 25: Backup and Restore.
- Capítulo 26: High Availability, Load Balancing, and Replication.
- Capítulo 30: Logical Replication.
- Appendix F: Additional Supplied Modules. F.36. Postgres_fdw